# UR3e_Motion_MTC — Motion Planning & Control Subsystem

## Purpose

The Motion Planning and Control is responsible for all the physical movements of the UR3e robotic arm. This subsystem receives a brick's position within the pickup zone, and the target placement position on the LEGO board. Utilising MoveIt Task Constructor, the UR3e plans and executes the full pick-and-place sequence while avoiding collisions with itself and the surrounding environment.

---

## System Requirements

| Component | Version |
|---|---|
| Operating System | Ubuntu 22.04 (via WSL2 on Windows 11) |
| ROS2 | Humble Hawksbill |
| MoveIt2 | 2.5.9 |
| MoveIt Task Constructor | 0.1.3 |
| RViz2 | 11.2.26 |
| WSL Version | WSL2 version 2.5.10.0 |
| Robot | Universal Robots UR3e |
| Gripper | OnRobot RG2 |

---

## Key Nodes, Topics, and Files

### Node

| Node | Description |
|---|---|
| `motion_node` | This ROS2 Node waits for pose data to be recieved from `ordered_pose_array`, then sets up the planning scene, planning and executing the pick-and-place task on the UR3e. |

### Subscribed Topics

| Topic | Type | Description |
|---|---|---|
| `ordered_pose_array` | `std_msgs/msg/Float64MultiArray` | get placement position. See Inputs section for format. Consists of 12 float values representing the brick pickup and target placement positions. See Inputs section for format. |

### Key Source Files

| File | Description |
|---|---|
| `src/ur3e_motion_mtc.cpp` | This is the main source file, containing the `MTCTaskNode` class, environment setup, pose subscriber, and `main()`. |
| `launch/ur3e_motion_mtc.launch.py` | This is the Launch File, loading the robot description, kinematics parameters, and planning configurations from `ur_onrobot_moveit_config`, and starts the `motion_node`. |

### External Dependencies

| Package | Purpose |
|---|---|
| `rclcpp` | ROS2 client library for nodes, publishers, and subscribers |
| `moveit_core` | Contains core MoveIt funtionality |
| `moveit_task_constructor_core` | Contains core MTC planning pipeline |
| `std_msgs` | `Float64MultiArray` message type for pose input |
| `tf2_geometry_msgs`, `tf2_eigen` | Quaternion and transform utilities |

## Inputs and Outputs

### Input — `ordered_pose_array`

A single `std_msgs/msg/Float64MultiArray` message containing 12 float64 values representing two poses in order:

```
[brick_x, brick_y, brick_z, brick_roll, brick_pitch, brick_yaw, target_x,
target_y, target_z, target_roll, target_pitch, target_yaw]
```

| Index | Value | Unit |
|---|---|---|
| 0 | brick x | metres |
| 1 | brick y | metres |
| 2 | brick z | metres |
| 3 | brick roll | radians |
| 4 | brick pitch | radians |
| 5 | brick yaw | radians |
| 6 | target x | metres |
| 7 | target y | metres |

| Index | Value | Unit |
|---|---|---|
| 8 | target z | metres |
| 9 | target roll | radians |
| 10 | target pitch | radians |
| 11 | target yaw | radians |

All positions must be in the `world` frame, which within the current setup is equivalent to the `base_link`. The makes the coordinates relative to the base of the UR3e.

The node waits for the the `ordered_pose_array` to be received before planning and executing movements on the UR3e.

### Output — Robot Motion

On receiving valid pose data the node executes this sequence on the UR3e:

```
1.  Open gripper
2.  Move arm to `camera_home`               (sampling planner)
3.  Move arm to above-brick position        (sampling planner, Connect stage)
4.  Descend toward brick                    (Cartesian planner, downward -Z)
5.  Close gripper around brick
6.  Attach brick to gripper in scene
7.  Lift brick upward                       (Cartesian planner, upward +Z)
8.  Move arm to above-target position       (sampling planner, Connect stage)
9.  Lower to place position                 (Cartesian planner, downward -Z)
10. Open gripper
11. Detach brick from gripper in scene
12. Retreat upward                          (Cartesian planner, upward +Z)
13. Return arm to `camera_home`             (sampling planner)
```

---

## How to Run

### Building

```
cd ~/ws_moveit2
colcon build --symlink-install --packages-select ur3e_motion_mtc
source install/setup.bash
```

### Launch Commands

**Terminal 1 — UR driver (connect to physical robot):**

```
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch ur_onrobot_control start_robot.launch.py \
  ur_type:=ur3e \
  onrobot_type:=rg2 \
  robot_ip:=0.0.0.0 \
  launch_rviz:=false
```

**Terminal 1 — UR driver (connect to Fake Hardware):**

```
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch ur_onrobot_control start_robot.launch.py ur_type:=ur3e
onrobot_type:=rg2 use_fake_hardware:=true launch_rviz:=false
```

**Terminal 2 — MoveIt + RViz:**

```
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch ur_onrobot_moveit_config ur_onrobot_moveit.launch.py ur_type:=ur3e
onrobot_type:=rg2 launch_rviz:=true
```

**Terminal 3 — Launch Motion Planning and Control Node:**

```
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch ur3e_motion_mtc ur3e_motion_mtc.launch.py
```

**Terminal 4 — Test Pose Publish:**

```
ros2 topic pub --once /ordered_pose_array std_msgs/msg/Float64MultiArray "{
  data: [0.3, 0.3, 0.02, 0.0, 0.0, 0.0,
         -0.25, 0.2, 0.03, 0.0, 0.0, 0.0]
}"
```

## Troubleshooting

If the MTC motion planning completes, but the execution fails, this may be an issue with the controllers.

**Controller List:**

First check which controllers are active and inactive.

```
ros2 control list_controllers
```

**Activate Gripper Trajectory Controller:**

```
ros2 control switch_controllers \
  --activate finger_width_trajectory_controller \
  --deactivate finger_width_controller
```

**Activate Scaled Joint Trajectory Controller:**

```
ros2 control switch_controllers \
  --activate scaled_joint_trajectory_controller \
  --deactivate joint_trajectory_controller
```

### Opening MoveIt Task Constructor in RViz

On the bottom left window, click the **Add** button.

Under `moveit_task_constructor_visualization`, select **Motion Planning Task**.

In **Display** under **Motion Planning Task**, change **Task Solution Topic** to `/solution`.

---

## Configurable Parameters

All parameters are set directly in `src/ur3e_motion_mtc.cpp`:

| Parameter | Location in code | Default | Description |
|---|---|---|---|
| **Planning Control** | | | |
| `PATH_COST_THRESHOLD` | `doTask()` | `30.0` | Maximum allowed path cost; planning retries if above this threshold |
| `MAX_PLANNING_ATTEMPTS` | `doTask()` | `20` | Number of full-task planning attempts before giving up |
| **Cartesian Motion Ranges** | | | |
| Approach min/max distance | `approach object` stage | `0.03 / 0.10 m` | Vertical descent range before grasping |
| Lift min/max distance | `lift object` stage | `0.03 / 0.10 m` | Vertical lift range after grasping |

| Parameter | Location in code | Default | Description |
|---|---|---|---|
| Retreat min/max distance | `retreat` stage | `0.03 / 0.10 m` | Vertical retreat range after placing |
| **Stage Timeouts** | | | |
| `move to pick` timeout | `stage_move_to_pick` | `5.0 s` | Max planning time to reach above the brick |
| `move to place` timeout | `stage_move_to_place` | `5.0 s` | Max planning time to reach above the place position |
| **Inverse Kinematics** | | | |
| Max IK solutions (grasp) | `ComputeIK` grasp wrapper | `16` | IK solutions explored for grasp pose |
| Min solution distance (grasp) | `ComputeIK` grasp wrapper | `0.5` | Min distance (rad) between consecutive grasp IK solutions |
| Max IK solutions (place) | `ComputeIK` place wrapper | `16` | IK solutions explored for place pose |
| Min solution distance (place) | `ComputeIK` place wrapper | `1.0` | Min distance (rad) between consecutive place IK solutions |
| **Velocity & Acceleration** | | | |
| Velocity scaling (sampling) | `sampling_planner` | `0.2` | Fraction of max joint velocity for sampling planner |
| Velocity scaling (Cartesian) | `cartesian_planner` | `0.2` | Fraction of max joint velocity for Cartesian stages |
| Acceleration scaling (sampling) | `sampling_planner` | `0.2` | Fraction of max joint acceleration for sampling planner |
| Acceleration scaling (Cartesian) | `cartesian_planner` | `0.2` | Fraction of max joint acceleration for Cartesian stages |
| Cartesian step size | `cartesian_planner` | `0.002 m` | Step size for discretised Cartesian planning |
| Cartesian jump threshold | `cartesian_planner` | `1.5` | Max allowed joint angle jump between Cartesian waypoints (rad) |

# Known Limitations and Assumptions

## Assumptions

- The pose data received from `ordered_pose_array` is in the `world` frame of reference.

- The pose data received from `ordered_pose_array` is an array of 12 float values representing the brick's pose and target pose, 6 vales for each.
- The node will only receive a single pick-and-place task per motion, and will only receive a new one once returning to the `camera_home` robot configuration.
- Two brick's will be distances far enough appart that the gripper can pick and place each brick using the brick's long axis.
- All required controllers have been actived before launching the `motion_node`.

## Known Limitations

- If planning fails, the node logs an error and exits. It does not retry automatically with different parameters.
- The workspace does not model the evironment of the actually UR3e. There is no placement board or other bricks, so the robot doesn't avoid collision with the environment yet. However, the approach and retreat cartesain movements around collisions with already placed bricks.
- The system cannot avoid dynamic obstacles within the environment.